

PRACTICAL - 8

Title

Simulation of Disk Scheduling Algorithms

Requirements

- Computer System
- Operating System: Windows / Linux / macOS
- Python 3.8 or higher
- Any code editor (VS Code / PyCharm / IDLE)

Theory

One of the responsibilities of an operating system is to use hardware efficiently. For hard disk drives (HDDs), this involves minimizing access time and maximizing data transfer rate.

Disk access time mainly consists of:

- **Seek Time:** Time taken by the disk arm to move the head to the required cylinder.
- **Rotational Latency:** Time taken for the disk to rotate the required sector under the head.

When multiple I/O requests are present, they are placed in a queue. Disk scheduling algorithms are designed to reduce seek time and improve disk I/O performance by deciding the order in which disk requests are serviced. Different algorithms provide different trade-offs between seek time, fairness, and starvation.

Disk Scheduling Algorithms

1. First Come First Serve (FCFS)

- Requests are serviced in the order they arrive in the queue
- Disk head moves sequentially from one request to the next
- Simple and easy to implement
- No starvation
- Fair scheduling
- May cause large head movement
- Poor average seek time

2. Shortest Seek Time First (SSTF)

- Selects the request closest to the current head position
- Reduces head movement and seek time
- Provides better performance than FCFS
- May cause starvation for distant requests
- More complex than FCFS

3. SCAN (Elevator Algorithm)

- Disk head moves in one direction servicing requests along the path
- After reaching the end of the disk, it reverses direction
- Provides better performance than FCFS
- Eliminates starvation
- Provides more uniform waiting time
- Head moves to disk end even if no requests exist

4. C-SCAN (Circular SCAN)

- Disk head moves in only one direction
- After reaching the end, it jumps to the beginning
- Requests are serviced again in the same direction
- Provides uniform waiting time
- Better fairness than SCAN
- Extra movement due to jump from end to beginning

Performance Comparison

The performance of disk scheduling algorithms is measured using **total seek time**.

- Lower seek time → Better performance
- SSTF usually gives minimum seek time
- SCAN and C-SCAN provide better fairness and avoid starvation

Code:

```
1 def fcfs(requests, head):
2     seek = 0
3     sequence = [head]
4
5     for r in requests:
6         seek += abs(head - r)
7         head = r
8         sequence.append(head)
9
10    return seek, sequence
11
12 def sstf(requests, head):
13     seek = 0
14     req = requests.copy()
15     sequence = [head]
16
17     while req:
18         closest = min(req, key=lambda x: abs(x - head))
19         seek += abs(head - closest)
20         head = closest
21         sequence.append(head)
22         req.remove(closest)
23
24    return seek, sequence
25
26 def scan(requests, head, disk_size):
27     seek = 0
28     sequence = [head]
29
30     left = [r for r in requests if r < head]
31     right = [r for r in requests if r >= head]
32
33     left.sort()
34     right.sort()
35
36     # moving towards right
37     for r in right:
38         seek += abs(head - r)
39         head = r
40         sequence.append(head)
41
42     # go to the end
43     seek += abs(head - (disk_size - 1))
44     head = disk_size - 1
45     sequence.append(head)
46
47     # moving left
48     for r in reversed(left):
```

```
49     seek += abs(head - r)
50     head = r
51     sequence.append(head)
52
53     return seek, sequence
54
55 def cscan(requests, head, disk_size):
56     seek = 0
57     sequence = [head]
58
59     left = [r for r in requests if r < head]
60     right = [r for r in requests if r >= head]
61
62     left.sort()
63     right.sort()
64
65     # right
66     for r in right:
67         seek += abs(head - r)
68         head = r
69         sequence.append(head)
70
71     # Go to end
72     seek += abs(head - (disk_size - 1))
73     head = disk_size - 1
74     sequence.append(head)
75
76     # Jump to beginning
77     seek += disk_size - 1
78     head = 0
79     sequence.append(head)
80
81     # Move right again
82     for r in left:
83         seek += abs(head - r)
84         head = r
85         sequence.append(head)
86
87     return seek, sequence
88
```

```

89 # main program
90 requests = list(map(int, input("Enter disk request sequence: ").split()))
91 head = int(input("Enter initial head position: "))
92 disk_size = int(input("Enter disk size: "))
93
94 fcfs_seek, fcfs_seq = fcfs(requests, head)
95 sstf_seek, sstf_seq = sstf(requests, head)
96 scan_seek, scan_seq = scan(requests, head, disk_size)
97 cscan_seek, cscan_seq = cscan(requests, head, disk_size)
98
99 print("\nDisk Scheduling Results ")
100 print("FCFS Seek Time:", fcfs_seek)
101 print("SSTF Seek Time:", sstf_seek)
102 print("SCAN Seek Time:", scan_seek)
103 print("C-SCAN Seek Time:", cscan_seek)
104
105 # comparison
106 best_seek = min(fcfs_seek, sstf_seek, scan_seek, cscan_seek)
107
108 print("\nBest Algorithm:", end=" ")
109 if best_seek == fcfs_seek:
110     print("FCFS")
111 elif best_seek == sstf_seek:
112     print("SSTF")
113 elif best_seek == scan_seek:
114     print("SCAN")
115 else:
116     print("C-SCAN")
117
118 # Show head movement sequence
119 print("\n HEAD Movement Sequences ")
120 print("FCFS:", fcfs_seq)
121 print("SSTF:", sstf_seq)
122 print("SCAN:", scan_seq)
123 print("C-SCAN:", cscan_seq)

```

Output:

```

C:\Users\user\AppData\Local\Programs\Microsoft VS Code> & c
Enter disk request sequence: 20 180 34 119 11 123 66 63 95
Enter initial head position: 70
Enter disk size: 200

Disk Scheduling Results
FCFS Seek Time: 753
SSTF Seek Time: 228
SCAN Seek Time: 317
C-SCAN Seek Time: 394

Best Algorithm: SSTF

HEAD Movement Sequences
FCFS: [70, 20, 180, 34, 119, 11, 123, 66, 63, 95]
SSTF: [70, 66, 63, 34, 20, 11, 95, 119, 123, 180]
SCAN: [70, 95, 119, 123, 180, 199, 66, 63, 34, 20, 11]
C-SCAN: [70, 95, 119, 123, 180, 199, 0, 11, 20, 34, 63, 66]

```

Conclusion

Disk scheduling algorithms were studied and implemented using a simulator. FCFS, SSTF, SCAN, and C-SCAN were compared based on total seek time. It was observed that SSTF gives minimum seek time, while SCAN and C-SCAN provide better fairness and more uniform waiting time.